This appendix accompanies the paper and is published in its artifact repository; its theorem, figure and section numbers continue those of the paper.

A PROOFS

This appendix proves the results of Secs. 3 to 5 in the order in which they appear. Throughout, instances are finite, and implication and the closure Σ^+ are taken in the finite sense. For an instance I and an entity e, we write $I(e)$ for the tuple of e in I.

A.1 Binary Updates

An update that inserts or deletes entities shows an inserted entity in I, and a deleted one in J, with an absent version, which violates no FD, needs no witness and witnesses nothing.

Proposition A.1 (Binary updates). *Let every IND $R[X] \subseteq S[Y]$ in Σ have a key K of S inside Y, and let no entity agree on K in J with another entity in I. Then the update is binary, and its precedence graph has an edge from v to u exactly when (i) the new tuple of u and the old tuple of v violate an FD in Σ, (ii) the new tuple of u has a witness under some IND only in the new tuple of v, or (iii) the old tuple of v has a witness under some IND only in the old tuple of u.*

Proof. An intermediate state shows every changed entity in its old or its new version, and every other entity in its only version. An FD relates two tuples, and two versions that violate it are not both old, since I is legal, nor both new, since J is legal, nor do they involve an unchanged entity, since its version occurs in I and in J. Hence the state satisfies the FDs exactly when it shows no pair of case (i). Consider an IND $R[X] \subseteq S[Y]$, a version t of an entity e, and two versions s and s' of entities w and w' with $s[Y] = s'[Y] = t[X]$. Then s and s' agree on K, so $w = w'$ by the hypothesis if one of them is old and the other new, and by the key K of I or J otherwise. Thus all candidate witnesses of t are versions of one entity w. If t is new, some version of w is a witness, since J is legal, and the state fails the IND at t only if it shows the old version of w while only the new one is a witness, which is case (ii) with $u = e$ and $v = w$, as w then changes. Symmetrically, if t is old, the state fails the IND at t only in case (iii) with $v = e$ and $u = w$. Hence a state is legal exactly when it shows no forbidden pair, that is, when no edge leads into the set of entities that it shows new from outside. □

The hypothesis is needed. If an update deletes an entity w' of S and inserts an entity w with the same value of K, while a changed entity e of R starts to reference this value, then the state that shows e new and w' deleted is illegal, although the states with only one of them are legal, and no precedence graph expresses this. For modification updates, the hypothesis holds whenever $K \subseteq Fz(S)$ or the update is fresh (Sec. 3.3), and inserted entities meet it when they take values of K that I does not contain.

A.2 Proofs for Sec. 3

We start with the invariant of the frozen closure.

Proof of Lem. 3.2. Let (I, J) be a modification update. We show by induction on the construction of M^* as a least fixpoint that $I(e)$ and $J(e)$ agree on $M^*(R)$ for every entity e of every relation R. The base case Fz holds by Def. 2.1. For the inductive step, let the rule add X_i to $M^*(R)$ through an IND $R[X] \subseteq S[Y]$ in Σ^+, where Y contains a key K of S with $K \subseteq M^*(S)$ and the attributes X_K of X at the positions of K lie in $M^*(R)$. Let e be an entity of R, and let $u = I(e)$ and $u' = J(e)$. Since I and J satisfy the IND, there are entities w and w' of S with $I(w)[Y] = u[X]$ and $J(w')[Y] = u'[X]$. By the induction hypothesis, $u[X_K] = u'[X_K]$ and $I(w)[K] = J(w)[K]$. Then $J(w)[K] = u[X_K] = u'[X_K] = J(w')[K]$, and $w = w'$ because J satisfies the key K. Consequently, $u'[X_i] = J(w)[Y_i] = I(w)[Y_i] = u[X_i]$ whenever $Y_i \in M^*(S)$. □

The invariant yields the claim made after Def. 3.3.

Lemma A.2. *Every constraint in $\mathcal{G}$ holds in every intermediate state of every modification update.*

Proof. Let (I, J) be a modification update, and let $L = I \oplus T$ for some $T \subseteq Ch$. Every tuple of L is $I(e)$ or $J(e)$ for an entity e, and $I(e)$ and $J(e)$ agree on M^* by Lem. 3.2. Consider first an FD $X \to A$ in $\mathcal{G}$, and let the tuples of two distinct entities e and f agree on X in L. If X contains a key $K \subseteq M^*$, then $I(e)$ and $I(f)$ agree on K, which contradicts the key K in I. If $X \cup \{A\} \subseteq M^*$, then $I(e)$ and $I(f)$ carry the values of the two tuples on $X \cup \{A\}$, so the two tuples agree on A because I satisfies the FD. Consider next an IND $R[X] \subseteq S[Y]$ in $\mathcal{G}$, and let t be the tuple of an entity e of R in L. If $Y \subseteq M^*(S)$, then I, J and L have the same projection on $S[Y]$, and $t[X]$ lies in the projection of I or of J. Otherwise, $X \subseteq M^*(R)$, and Y contains a key K of S with $K \subseteq M^*(S)$. Then $t[X] = I(e)[X] = J(e)[X]$, and there are entities w and w' of S with $I(w)[Y] = t[X] = J(w')[Y]$. By Lem. 3.2, $J(w)[K] = I(w)[K] = J(w')[K]$, so $w = w'$. The tuple of w in L is a witness for t. □

Proof of Thm. 3.5. Every intermediate state is a finite instance that satisfies $\mathcal{G}$ by Lem. A.2. Since $\mathcal{G}$ implies Σ, the state satisfies Σ. Hence every modification update is order-insensitive, and in particular executable. □

For FDs we use a classical fact: an instance of a relation with two tuples satisfies a set of FDs if and only if the attributes on which the tuples agree, their *agree set*, form a closed set. For a relation R with attributes U, we write $N = U \setminus Fz(R)$ for its mutable attributes. Two tuples t_1 and t_2 *swap their mutable parts* if they are replaced by t_1', which agrees with t_1 on Fz and with t_2 on N, and by t_2', defined symmetrically. The agree set of t_1' and t_2' is that of t_1 and t_2, and the agree set of t_1' and t_2 is the union of N and the frozen part of the agree set of t_1 and t_2. The following lemma justifies the remark after Thm. 3.6 that order-sufficiency freezes every key.

Lemma A.3. *Let Σ contain only FDs and satisfy condition (4) of Thm. 3.6. Then every minimal key of every relation is frozen.*

Proof. Assume that a minimal key K of a relation R has a mutable attribute, and let $C = (K \cap Fz)^+$. Since K is minimal, $C \neq U$, and therefore $PK \not\subseteq C$. Let $Z = (C \cap Fz) \cup N$. Then $Z \supseteq K$, so $Z^+ = U$, whereas $PK \not\subseteq Z$ because $PK \subseteq Fz$. Consequently, Z is not closed, and some elementary FD $Y \to b$ in Σ^+ has $Y \subseteq Z$ and $b \notin Z$. By condition (4), $Y \subseteq Fz$, so $Y \subseteq C$ and $b \in C$. Since $N \subseteq Z$, also $b \in Fz$, and thus $b \in C \cap Fz \subseteq Z$, a contradiction. □

Proof of Thm. 3.6. Since Σ contains no INDs, $M^* = Fz$.

(4)⇒(3). Let $X \to A$ be an elementary FD in Σ^+. If X is a superkey, it contains a key that determines A, and since no proper subset of X determines A, X is itself a key. By (4), X is a frozen key, so the FD belongs to $\mathcal{G}$. Otherwise, $X \cup \{A\} \subseteq Fz$ by (4), and the FD belongs to $\mathcal{G}$ again. Every nontrivial FD of Σ follows from an elementary FD of Σ^+ with a smaller left side, so $\mathcal{G} \models \Sigma$.

(3)⇒(2) is Thm. 3.5, and (2)⇒(1) holds because every prefix state is an intermediate state.

(1)⇒(4). Let $X \to A$ be an elementary FD over R in Σ^+ that violates (4). We build a modification update (I, J) in which R holds two tuples, every other relation is empty, and either intermediate state with one changed entity violates Σ. In each case, I holds two tuples whose agree set C is closed and differs from U, so they differ on PK and are distinct entities. *Case 1: X is a superkey.* Then X is a minimal key with a mutable attribute, as shown above. Let $C = (X \cap Fz)^+$, and let J arise by swapping the mutable parts. Both entities change, since a mutable attribute of X lies outside C by the minimality of X. The intermediate state $\{t_1', t_2\}$ agrees on $(C \cap Fz) \cup N \supseteq X$ but not on PK, which violates the key X. *Case 2: X is not a superkey and A is mutable.* Let $C = X^+$, and let J give both tuples the same new value on A. The agree set of J is C again, and that of $\{t_1', t_2\}$ is $C \setminus \{A\} \supseteq X$, which violates $X \to A$. *Case 3: X is not a superkey, A is frozen, and some $x \in X$ is mutable.* Let $C = (X \setminus \{x\})^+$. Since the FD is elementary, $A \notin C$ and $x \notin C$. Let J arise by swapping the mutable parts, which changes both entities because $x \notin C$. The intermediate state $\{t_1', t_2\}$ agrees on $(C \cap Fz) \cup N \supseteq X$ but not on the frozen attribute A. The three cases cover every violation of (4), and the intermediate state $\{t_1, t_2'\}$ is symmetric in each. Hence the update is not executable. □

The proof also yields the polynomial test claimed in Thm. 3.6. For a set Z of attributes of R, let $Z' = Z \cup ((Z \cap Fz)^+ \cap Fz)$. The closure of Z under $\mathcal{G}$ is U if $(Z \cap Fz)^+ = U$, and Z' otherwise. Indeed, in the first case $Z \cap Fz$ contains a frozen key, and in the second case Z' is closed under both kinds of FDs in $\mathcal{G}$, since $(Z' \cap Fz)^+ = (Z \cap Fz)^+$ contains no key. Deciding $\mathcal{G} \models \Sigma$ takes one closure under Σ for every FD of Σ.

For fresh updates, we use that the left sides of elementary FDs consist of attributes that occur on left sides of Σ.

Proof of Prop. 3.8. Let $X \to A$ be an elementary FD in Σ^+, and let $x \in X$ occur on no left side of an FD in Σ. Then x fires no FD of Σ, so the closure of X is the closure of $X \setminus \{x\}$ together with x, and $A \neq x$ lies in the closure of $X \setminus \{x\}$, which contradicts elementarity. Hence every attribute of X receives only new values from a fresh update, unless the update leaves it unchanged.

Assume the condition, and let the tuples of two distinct entities violate Σ in an intermediate state $I \oplus T$. They violate an elementary FD $X \to A$ of Σ^+, and since I and J satisfy Σ, one of them is the new tuple $J(u)$ of some $u \in T$ and the other the old tuple $I(v)$ of some $v \in Ch \setminus T$. For every $x \in X$, the update leaves $J(u)[x] = I(u)[x]$, since a new value of x would differ from $I(v)[x]$. Hence $I(u)$ and $I(v)$ agree on X, so X is no superkey, and they agree on A, so $J(u)[A] \neq I(u)[A]$ and A is mutable, which contradicts the condition. Consequently, every fresh update is order-insensitive.

Conversely, let $X \to A$ be an elementary FD in Σ^+ whose left side is no superkey and whose right side is mutable. Case 2 of the proof of Thm. 3.6 takes two tuples whose agree set is X^+ and gives both a value of A that occurs nowhere in I. The update writes only this value, so it is fresh, and neither intermediate state with one changed entity satisfies $X \to A$. □

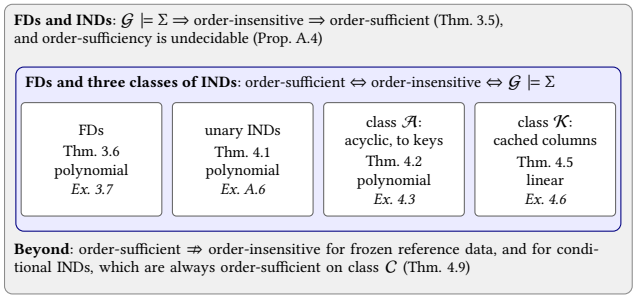


Figure 8: Order-Sufficiency by Class of Constraints

A.3 Proofs for Sec. 4

Fig. 8 summarizes the results of Sec. 4 by class of constraints.

Order-sufficiency inherits the undecidability of implication.

Proposition A.4 (Undecidability). *For FDs and INDs, it is undecidable whether Σ is order-sufficient under Fz, and whether Σ is order-insensitive under Fz, already when the workload modifies a single attribute and the frozen attributes are exactly the prime attributes.*

The proof needs a source problem without FDs whose left side is empty.

Proof of Prop. A.4. Finite implication of an FD by FDs and INDs remains undecidable when every FD has a nonempty left side, as Abiteboul et al. note in their proof of this result [1, p. 200]. Let Σ be such a set over relations with *original* attributes, and let $X \to A$ be an FD over R with $A \notin X$. We may assume $X \neq \emptyset$. Indeed, the instance that holds one tuple of zeros in every relation satisfies Σ, and so does its disjoint union with a copy of ones, since FDs with nonempty left sides and INDs survive disjoint unions. Hence Σ implies no nontrivial FD with an empty left side, and the restriction to $X \neq \emptyset$ keeps the problem undecidable.

We add to every relation S a surrogate primary key ID_S, add to R a new attribute V with the FD $XV \to A$, and freeze every attribute except V. Call the result Σ'. Assume first that $\Sigma \models X \to A$. Every intermediate state L of a modification update (I, J) agrees with I outside V. It satisfies every constraint of Σ' that does not mention V, as I does, and its projection on the original attributes satisfies Σ and therefore $X \to A$. Consequently, L satisfies $XV \to A$, and Σ' is order-insensitive. Assume next that $\Sigma \not\models X \to A$, and let I_0 be a finite instance of Σ with tuples t_1 and t_2 that agree on X but not on A. We extend I_0 with distinct surrogate values, give t_1 and t_2 the V-values 1 and 2, and give every other tuple a new V-value. The result I satisfies Σ', and so does the instance J that swaps the V-values of t_1 and t_2. Either intermediate state with one changed entity makes t_1 and t_2 agree on XV but not on A. Hence Σ' is order-sufficient, and order-insensitive, exactly when $\Sigma \models X \to A$, which shows the first claim.

For the second claim, we add to every relation S and every original attribute B of S a frozen attribute N_B with the FD $BN_B \to ID_S$,

and to R the FDs $XN_C \to ID_R$ for every original attribute C of R. The argument above carries over, since the new FDs do not mention V and pairwise distinct N-values satisfy them. It remains to show that the prime attributes are exactly the frozen ones. The surrogate ID_S is a key. Every original B and its N_B form a minimal key, for two reasons. Firstly, B alone is no superkey, as witnessed by two tuples that agree on all original attributes and on V and differ elsewhere. Secondly, N_B alone is no superkey, as witnessed by the disjoint union above, extended with equal N-values and distinct V-values: its two tuples per relation differ on every original attribute, which satisfies the new FDs because $X \neq \emptyset$. Finally, V is not prime. The first witness shows that every superkey contains a surrogate or an N-attribute. Let K be a minimal key of R with $V \in K$. Then K contains no surrogate, since the surrogate alone is a key, and hence some N_C. Since $K \setminus \{V\}$ is no superkey, some instance has tuples $p \neq q$ that agree on $K \setminus \{V\}$, and hence on N_C. By $XN_C \to ID_R$, the tuples p and q differ on X. Giving every tuple of R a distinct V-value, except that q takes the value of p, preserves $XV \to A$ and every other constraint. The resulting instance has two tuples that agree on K, a contradiction. □

For unary INDs, the key step permutes values column by column.

Lemma A.5. *Let Σ consist of FDs and unary INDs, let I satisfy Σ, and let R be a relation. For every mutable attribute B of R, let π_B be a permutation of the values in column B of $I(R)$. Replacing every tuple t of R by the tuple that agrees with t on $Fz(R)$ and takes the value $\pi_B(t[B])$ on every mutable B yields an instance J, and (I, J) is a modification update.*

Proof. Entities and frozen values are unchanged. Every π_B is injective, so every two tuples of R keep their agree set, and J satisfies the FDs. Every column keeps its set of values, and J satisfies the unary INDs. □

Proof of Thm. 4.1. Since $Fz \subseteq M^*$, a unary IND with a frozen right side belongs to $\mathcal{G}$.

(4)⇒(3). The first part of (4) places every nontrivial unary IND of Σ^+ in $\mathcal{G}$. The second part places every elementary FD of Σ^+ in $\mathcal{G}$, as in the proof of Thm. 3.6. Hence $\mathcal{G} \models \Sigma$. Thm. 3.5 gives (3)⇒(2), and (2)⇒(1) is immediate. (1)⇒(5) holds because every write order passes through a prefix state that contains the new tuple of exactly one of two such entities.

(5)⇒(4). Assume that (4) fails. In each of the following cases, we take a finite instance I of Σ, obtain J by Lem. A.5, and exhibit two changed entities that must become visible together. Every intermediate state that contains the new tuple of exactly one of them violates a constraint of Σ^+, and hence Σ. We describe the state in which the first entity is new, and the other state is symmetric. We use that the disjoint union of an instance of Σ with a copy in which all values are renamed is again an instance of Σ, since the FDs have nonempty left sides. *Case 1: a nontrivial unary IND $R[A] \subseteq S[B]$ in Σ^+ has a mutable right side.* Let I be the disjoint union of the instance with one tuple of zeros per relation and its copy with $0'$. Transpose 0 and $0'$ in column $S.B$. The changed entities are the two tuples of S. If only the first is new, column $S.B$ holds only $0'$, whereas column $R.A$ still holds 0. *Case 2: some elementary FD $X \to A$ in Σ^+ has a superkey X with a mutable attribute.* Then X is a minimal key, and $X \cap Fz$ is no superkey. Let I hold distinct tuples t_1 and t_2 of R that agree on $X \cap Fz$, and transpose $t_1[B]$ and $t_2[B]$ in every mutable column B. Both tuples change, since they differ on a mutable attribute of the key X. The new t_1 agrees with the old t_2 on X but not on PK, which violates the key X. *Case 3: some elementary FD $X \to A$ in Σ^+ has a non-superkey X and a mutable A.* Let I hold distinct tuples t_1 and t_2 that agree on X, and hence on A with a value a, together with a renamed copy that carries a value b in column A. Transpose a and b in column A. The new t_1 agrees with the old t_2 on X but not on A. *Case 4: some elementary FD $X \to A$ in Σ^+ has a non-superkey X with a mutable x, and A is frozen.* Since $X \setminus \{x\}$ does not determine A, let I hold tuples t_1 and t_2 that agree on $X \setminus \{x\}$ but not on A, and transpose $t_1[B]$ and $t_2[B]$ in every mutable column B. Both tuples change, since they differ on x, as X determines A. The new t_1 agrees with the old t_2 on X but not on A. The four cases cover every violation of (4).

Polynomial time. By the axiomatization of finite implication for FDs and unary INDs, its cycle rules derive only unary FDs and unary INDs [1, Thm. 9.4.7]. Consequently, the FDs in Σ^+ are generated by the FDs of Σ and polynomially many unary FDs, all computable in polynomial time [22]. The first part of (4) inspects polynomially many unary INDs, and the second part is the test after the proof of Thm. 3.6, applied to these generators. □

The following example shows why condition (4) must be computed under finite implication.

Example A.6 (Finite implication matters). Let $R(K, X, A)$ have the primary key K, the unary IND $R[K] \subseteq R[X]$ and the declared FD $X \to A$, and let $Fz = \{K, X\}$. In every finite instance, X takes at least as many values as K, so X is a key, condition (4) holds, and every modification of A is order-insensitive. Under unrestricted implication, X is not a key, and the test would wrongly reject the schema.

For the class $\mathcal{A}$, the completion of a two-tuple core rests on the following chase.

Lemma A.7. *Let F be a set of FDs over a relation S, let K be a minimal key of S under F, and let V be a finite set of values for K. Some finite relation over S satisfies F and has exactly V as its projection on K.*

Proof. For every value in V, create a tuple that takes this value on K and distinct nulls elsewhere. Chase with F: whenever two tuples agree on Z and $Z \to E \in F$, merge their values on E. For tuples u and v, let $M(u, v)$ be the attributes outside K on which they carry merged nulls, and let $A(u, v)$ be the attributes of K on which they agree. We show the invariant that $K \setminus \{B\} \to M(u, v)$ follows from F for all u, v and $B \in K$ with $u[B] \neq v[B]$. It holds initially, since M is empty. Let a step merge the E-values of p and q, which agree on Z with $Z \to E \in F$, and let it newly merge x and y, where x was merged with p and y with q on E before. If $x[B] \neq y[B]$, two neighbours in the sequence x, p, q, y differ on B. For the pairs x, p and q, y, the invariant already gives $K \setminus \{B\} \to E$. For the pair p, q, we have $Z \subseteq A(p, q) \cup M(p, q)$ and $A(p, q) \subseteq K \setminus \{B\}$, so the invariant gives $K \setminus \{B\} \to Z$ and therefore $K \setminus \{B\} \to E$. Now assume that the chase must merge two distinct values on some $B \in K$, because p and q agree on Z with $Z \to B \in F$ and

$p[B] \neq q[B]$. The invariant gives $K \setminus \{B\} \to Z$, hence $K \setminus \{B\} \to B$, which contradicts the minimality of K. Hence the chase never merges values of K, and replacing every class of nulls by a new value yields the relation. □

Proof of Thm. 4.2. For a relation R, let F_R be its declared FDs.

(4)⇒(3). By Lem. A.3, applied to every F_R, every minimal key of every relation is frozen. The right side of every IND is such a key, so every IND belongs to $\mathcal{G}$. As in the proof of Thm. 3.6, every elementary FD of F_R^+ belongs to $\mathcal{G}$, since a key under F_R is a key under Σ. Consequently, $\mathcal{G} \models \Sigma$. Thm. 3.5 gives (3)⇒(2), and (2)⇒(1)⇒(5) is immediate.

(5)⇒(4). Assume that F_R violates condition (4) of Thm. 3.6. The proof of that theorem yields tuples t_1, t_2 and t'_1, t'_2 over R such that both pairs satisfy F_R and either intermediate state with one changed entity violates F_R. We complete both pairs with the same tuples W. Process the relations that the IND graph reaches from R in topological order. For such a relation S, collect the values that the tuples built so far, including t_1, t_2, t'_1, t'_2, carry on the left sides of the INDs into S. All these INDs reference the same minimal key of S, so Lem. A.7 builds the tuples of S. The process terminates because the IND graph is acyclic, and it adds no tuple to R. All other relations stay empty. Then $\{t_1, t_2\} \cup W$ and $\{t'_1, t'_2\} \cup W$ satisfy Σ, contain the same entities, and agree on frozen values. This update has exactly two changed entities and is not executable. □

For the class $\mathcal{K}$, both directions work IND by IND.

Proof of Thm. 4.5. (4)⇒(3). The primary keys are frozen keys and belong to $\mathcal{G}$. Let $R[PD] \subseteq S[LC]$ satisfy (4). If $C \subseteq Fz$, its right side lies in $Fz \subseteq M^*$. Otherwise, $P \subseteq Fz$ and every D_i is frozen or copies a frozen C_i. In the latter case, the rule of Def. 3.1 puts D_i into $M^*(R)$, because L is frozen and P sits at its positions. The left side then lies in $M^*(R)$, the right side contains the frozen key L, and the IND belongs to $\mathcal{G}$. Hence $\mathcal{G} \models \Sigma$. Thm. 3.5 gives (3)⇒(2), and (2)⇒(1) is immediate.

(1)⇒(4). Let $\rho = R[PD] \subseteq S[LC]$ violate (4). Then $C \not\subseteq Fz$, and either some position i has a mutable C_i and a mutable D_i, or $P \not\subseteq Fz$ and every mutable C_i has a frozen D_i. Note that $R \neq S$, since the IND graph is acyclic. Let π, π_1, π_2, a and b be new constants. *Core of the first case.* An entity r of R points to π, and the entity s of S with key π has $s[C_i] = r[D_i] = a$ in I and b in J. All other copies and sources of ρ are 0. Either intermediate state with one changed entity leaves r without a witness. *Core of the second case.* Choose a mutable C_j, whose copy D_j is frozen. Two entities r_1 and r_2 of R differ on an attribute of $PK(R)$ outside P, which exists because P does not contain $PK(R)$. Their pointers π_1 and π_2 differ exactly on the mutable attributes of P, and $r_1[D_j] = a$ and $r_2[D_j] = b$. The entities s_1 and s_2 of S with keys π_1 and π_2 have $s_1[C_j] = a$ and $s_2[C_j] = b$. In J, r_1 points to π_2, r_2 points to π_1, $s_1[C_j] = b$ and $s_2[C_j] = a$. All other copies and sources of ρ are 0. Whichever of the four entities is written first, some entity of R loses its witness: a new r_1 or r_2 finds no matching tuple yet, and a new s_1 or s_2 strips the old r_1 or r_2. *Completion.* The constants π, π_1 and π_2 have pairwise distinct new components, and the primary keys of core tuples consist of new constants. Every other attribute of a core tuple that occurs in a pointer gets a new constant, and every remaining attribute gets 0. Every relation gets a base tuple of zeros. For every core or support tuple and every IND $Q[P'D'] \subseteq T[L'C']$ of its relation other than ρ, the relation T needs a witness whose key is the value of the tuple on P' in I or J. Unless a core tuple has this key, T gets a support tuple with this key and 0 elsewhere, and the process terminates because the IND graph is acyclic. Such a support tuple is a witness, since the copies of the referencing tuple are 0 and the sources of T lie outside its key. Core tuples lie in R and S, and support tuples never reach R. A tuple can need the key π, π_1 or π_2 of S only through an IND without copies. Indeed, the other pointers of R into S differ from P in some position, and a support tuple carries 0 outside its key, so the pointer of an IND with copies, which leaves the primary key, yields a key with a component 0. The core tuple with this key is then a witness, since there are no copies to match. Hence every relation keeps distinct keys, every core violation persists, and the update is not executable. □

Both restrictions on pointers in $\mathcal{K}$ are needed. Firstly, let the relation $R(A, B, D)$ have the primary key A, let $S(K_1, K_2, C)$ have the primary key K_1K_2, and let only B and C be mutable under $R[ABD] \subseteq S[K_1K_2C]$, whose pointer contains $PK(R)$. Condition (4) fails, but an entity of S can only be referenced by the entity of R whose key it carries. Hence writing first the entities of S that gain their referencing entity, then the entities of R, and last the other entities of S executes every modification update. Yet an update that moves an entity of R away from an entity of S and changes that entity is not order-insensitive. Secondly, let $T(K_1, K_2, V)$ be frozen with the primary key K_1K_2, let $R(K, A, B, D)$ have the primary key K and the frozen attributes B and D, and let $S(L, E)$ have only its primary key L frozen. The INDs are $R[AD] \subseteq S[LE]$, $R[AB] \subseteq T[K_1K_2]$ and $T[K_1V] \subseteq S[LE]$, and the pointer of the third lies inside $PK(T)$. Condition (4) fails for the first IND, since A and E are mutable. Yet the key L makes the three INDs imply $R[ABD] \subseteq T[K_1K_2V]$, whose right side is frozen, and this IND and the third one imply the first. Hence $\mathcal{G} \models \Sigma$, and the schema is order-insensitive by Thm. 3.5. As in Ex. A.9, the frozen relation T fixes the values that referenced entities of S carry.

Proof of Prop. 4.7. Let (I, J) be a fresh modification update, let $\rho = R[PD] \subseteq S[LC]$ be an IND of Σ, and let r be an entity of R. Let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. They exist and are unique, since L is a frozen primary key, and every intermediate state needs one of them as the witness of r under ρ.

Assume that every position i has $C_i \in Fz$ or $D_i \in Fz$. Then s_J keeps its sources. Indeed, for a mutable C_i the copy D_i is frozen, so $J(s_J)[C_i] = J(r)[D_i] = I(r)[D_i]$ is a value of C_i in I, which a fresh update writes into no source. Hence the new tuple of r has its witness in every intermediate state, and the old tuple of r lacks it exactly when s_I is new and has changed a source of ρ. Since the primary keys are frozen and are the only FDs, an intermediate state $I \oplus T$ is legal exactly when T contains every such r whenever it contains s_I. Each of these requirements leads from an entity of R to an entity of S along an edge of the IND graph, so writing the relations in a topological order of the IND graph, referencing relations first, keeps every prefix state legal. If condition (4) of Thm. 4.5 holds as well, every modification update is order-insensitive by that theorem.

Conversely, let some position i have a mutable C_i and a mutable D_i. The core of the first case in the proof of Thm. 4.5, completed as there, changes only r and s and writes the new constant b, so it is a fresh update with two changed entities that is not executable. Finally, let every position satisfy the condition, and let condition (4) of Thm. 4.5 fail for ρ. Then $P \not\subseteq Fz$ and some C_j is mutable. An entity r of R points to π_1 in I and to π_2 in J, where π_1 and π_2 differ exactly on the mutable attributes of P, and the entities s_1 and s_2 of S with these keys have $s_1[C_j] = s_2[C_j] = r[D_j] = a$ in I. Besides r, only s_1 changes, to $s_1[C_j] = b$, and all other copies and sources of ρ are 0. After the same completion, the update is fresh, since b and π_2 are new, and writing s_1 before r strips the old r of its witness. □

A bounded SAT search confirms Prop. 4.7, including the order of the relations, on 17,436 pairs of 403 random schemas of class $\mathcal{K}$ and freezes, with up to five entities and five values, and finds a witness with two changed entities wherever a condition fails.

Unique keys. Unique keys besides the primary key can pin pointers. Indeed, if a referencing entity holds frozen copies of attributes that form a unique key of S, then it can never move to another entity of S, since that entity would have to carry the same values. Let Σ belong to $\mathcal{K}$, except that its relations may also declare unique keys, which no IND references. For a set M of attributes and an IND $\rho = R[PD] \subseteq S[LC]$, let $X_\rho(M)$ be the least set of attributes of S that contains every attribute of LC whose counterpart in PD lies in M, and the copies of every IND $S[P'D'] \subseteq T[L'C']$ with $P' \subseteq X_\rho(M)$. Let Fz^* be the least set $M \supseteq Fz$ that contains the pointer P of every IND ρ for which $X_\rho(M)$ contains a unique key of S. It exists, since $X_\rho(M)$ grows with M, and it results from adding such pointers to Fz until none is missing. The iteration matters when pointers overlap. Once Fz^* contains the pointer of ρ, the attributes that it shares with the pointer of another IND ρ' out of R enter M, and their counterparts may complete a unique key in $X_{\rho'}(M)$.

Proposition A.8 (Unique keys). *For such schemas, Prop. 4.7 holds with Fz^* in place of Fz.*

Proof. Let (I, J) be a fresh modification update under Fz. Firstly, every intermediate state satisfies the unique keys. Indeed, let u and v be distinct entities of a relation with a unique key U, and consider the old tuple of u next to the new tuple of v. If v keeps its values on U, the two tuples differ on U since I is legal, and otherwise v writes into U a value that is new in its column, which u does not carry in I. Secondly, (I, J) changes no attribute of Fz^*. We show by induction along the iteration that if (I, J) changes no attribute of a set $M \supseteq Fz$, then it changes no pointer that the next step adds. Indeed, let $X_\rho(M)$ contain a unique key U of S, let r be an entity of R, and let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. Let A be an attribute of LC whose counterpart B in PD lies in M. Then $J(s_J)[A] = J(r)[B] = I(r)[B] = I(s_I)[A]$. If A lies in the key L, which is frozen, then $I(s_J)[A] = J(s_J)[A]$. If A is a source, then $J(s_J)[A]$ is a value of A in I, so the fresh update has not written it, and again $I(s_J)[A] = J(s_J)[A]$. The copies of an IND $S[P'D'] \subseteq T[L'C']$ are determined by P', since L' is the primary key of T. Hence s_I and s_J agree on $X_\rho(M)$ in I, so on U, and $s_I = s_J$, so r keeps its pointer P. The fresh modification updates under Fz and under Fz^* thus coincide, and the unique keys hold in every intermediate state. Then the first part of the proof of Prop. 4.7 applies with Fz^* in place of Fz. If condition (4) holds for Fz^* as well, then either s_I and s_J keep their sources, or $s_I = s_J$ and every position i has C_i or D_i unchanged, so every combination of old and new tuples of r, s_I and s_J satisfies ρ, and every intermediate state is legal.

Conversely, the counterexamples of the proof of Prop. 4.7, taken under Fz^*, are fresh modification updates under Fz, and they carry over once the completion gives every support tuple its own new constants outside its key and every copy the value of its source. Two tuples of one relation then agree on an attribute only where the construction forces them to. This happens only for the old and the new witness of an entity r that moves under some IND ρ whose pointer P is not contained in Fz^*. The two witnesses agree on the attributes of L whose counterparts in P lie in Fz^*, on the sources whose copies lie in Fz^*, and on the copies that these attributes determine, that is, on $X_\rho(Fz^*)$, and on nothing else. Since Fz^* is a fixpoint, $X_\rho(Fz^*)$ contains no unique key of S. Hence the two witnesses differ on an attribute of every unique key, and every counterexample respects the unique keys. □

The bounded SAT search confirms Prop. A.8 on 4,172 pairs of random schemas and freezes with one or two unique keys of one or two non-key attributes, where six pairs need seven entities for their witness. Under Fz in place of Fz^*, 157 of these pairs would be predicted wrongly. Pointers that overlap need the iteration. On 8,498 pairs of 144 schemas in which two INDs out of one relation share pointer attributes, with unique keys on sources, on sources together with key attributes and on copies, the search finds a counterexample for every pair that Prop. A.8 rejects, with up to eight entities, and none for the others with up to five. Computing every set X_ρ under Fz alone, without the iteration, rejects 28 pairs wrongly.

Conditional INDs. Part (2) of Thm. 4.9 restricts the pointers in two ways: (a) every CIND with a condition on S has a pointer $P \not\subseteq PK(R)$, and (b) if such a CIND has no condition on R, it shares its pointer and S with no other CIND. Without these restrictions, a CIND can imply another, and the condition of part (2) remains sufficient but is no longer necessary. For (b), let $Project[gid] \subseteq Group[gid; arch \in \{0\}]$ join the CIND of Sec. 4.1, which it implies, and let $Project.gid$ be frozen while both flags are mutable. The condition fails for the CIND of Sec. 4.1, which tests the mutable flag of projects, yet every group with a project keeps $arch = 0$ in I and in J, so every modification update is order-insensitive. For (a), let $R[P] \subseteq T[K]$ and $T[K] \subseteq S[L; g \in B]$ with $K = PK(T)$ imply $\rho = R[P] \subseteq S[L; g \in B]$, and let P and g be mutable. The condition fails for ρ, yet an entity of S to which an entity of R points in I or in J shares its key with an entity of T in both instances, which needs it to meet $g \in B$ in both, so every modification update is order-insensitive. On CINDs, we call a modification update *fresh* if, beyond Sec. 3.3, every value that it writes into an attribute that a CIND tests on its right side is new in its column. The proof reads the precedence graph of Prop. A.1 through enabled and disabled entities.

Proof of Thm. 4.9. (1) The argument of Prop. A.1 applies to CINDs, since a condition only selects the tuples that need a witness and the tuples that can serve as one, and $L = PK(S)$ identifies one

entity. Hence the update is binary. An edge from v to u either leaves a referenced entity v whose new tuple alone serves the new tuple of u, so that the update enables v, or enters a referenced entity u whose old tuple alone serves the old tuple of v, so that the update disables u. No update both enables and disables an entity e of S. Indeed, both tuples of e would be present, with the old value of g in $B_2 \setminus B_1$ and the new one in $B_1 \setminus B_2$ for the sets B_1 and B_2 of two CINDs into S, which form a chain. Every edge thus leaves an enabled entity or enters a disabled one, and it joins two relations that the acyclic IND graph orders strictly. Consider the order of part (1) and an edge from v to u. If the edge leaves an enabled v, then v lies in the first group, and u lies in a later group or is an enabled entity of a relation that references the relation of v, which the first group writes later. Symmetrically, if the edge enters a disabled u, then u lies in the last group, after every entity of the earlier groups and after the disabled entities of the relations that reference the relation of u. No edge joins two entities of the middle group. Hence the order respects every edge, every prefix is closed under predecessors, and every prefix state is legal. Part (1) does not use the restrictions of part (2).

(2) Let every CIND that tests a mutable g have a frozen pointer and test no mutable f. A modification keeps every entity present, so a CIND that tests a frozen g or no attribute of S yields no edge. For a CIND that tests a mutable g, a tuple of R that needs a witness in I needs it in J at the same entity, and conversely, so this entity meets the condition on S in both versions, and the CIND yields no edge either. Hence the precedence graph has no edge, and every intermediate state is legal. This direction does not use the restrictions either.

Conversely, let $\rho = R[P; f \in A] \subseteq S[L; g \in B]$ test a mutable g, and let $P \not\subseteq Fz$ or f be mutable. For every relation whose CINDs test an attribute on the right side, fix a value b_1 in the least set of its chain, and for S also a value b_0 outside every set of its chain. The update changes two entities, r of R and s of S, where s carries $g = b_0$ in I and $g = b_1$ in J, and both keep all other values. If $P \not\subseteq Fz$, then r meets $f \in A$, if ρ tests f, in I and J, and it points to s in J and to an unchanged entity s' with $g = b_1$ in I, whose key differs from that of s exactly at the mutable attributes of P. If $P \subseteq Fz$, then f is mutable, r points to s in I and J, and f changes from a constant outside every set that a CIND tests f against to a value in A. The keys of s and s' and every other attribute of r, s and s' that no CIND tests carry new constants, and every other tested attribute on the left carries a constant outside all sets of its CINDs. Top-down along the acyclic IND graph, the completion then gives every tuple that needs a witness in I or J an unchanged entity at the key that it points to, with b_1 in the tested attribute, if any, and new constants in its other attributes. Hence every pointer of a completion entity into S under a CIND with a condition on S contains an attribute outside the key by restriction (a), whose value differs from the new constants in the key of s, so it never points to s.

I and J are legal. Every completion entity and s' meet every condition on their right side, since b_1 lies in every set of their chains, and they exist in both instances. The new tuple of s meets every condition on S as well. In J, the new tuple of r needs at s at most these conditions, and in I it points to s' if $P \not\subseteq Fz$. If $P \subseteq Fz$, then the old tuple of r meets no condition on R, so it needs at s only the CINDs without a condition on R, which by restriction (b) test no attribute of S, since they share the pointer and S with ρ. Hence no tuple of I needs the old tuple of s to meet a condition on S, and every other demand of I and J is met by construction. The intermediate state that shows only r new is illegal, since the new tuple of r meets $f \in A$ and points to the old tuple of s, which fails $g \in B$. The update is thus not order-insensitive.

The reverse update (J, I), with b_0 new in the column of g and, if $P \subseteq Fz$, the constant of f in I new in its column, is fresh, since it writes no key and no other tested value. It disables s, and its intermediate state that shows only s new is illegal, so part (2) also holds for fresh updates. □

For modification updates, the proof uses only that L identifies one entity in every state. It therefore applies also when L is a unique key of S that no update modifies, as for the flags that Sentry sets for accounts provisioned by an identity provider (Sec. 6.2).

Beyond the exact classes. Outside the three classes, write order can suffice for INDs although the condition of Thm. 3.5 fails, as the following example shows for frozen reference data.

Example A.9 (Frozen reference data). Let $Ref(L, Q)$ be a reference table with primary key L that the workload never modifies. Let $R(N, P, D)$ have the primary key N, the mutable pointer P and the frozen copy D, and let $S(L, C)$ have the primary key L and the mutable source C. The INDs $R[P, D] \subseteq Ref[L, Q]$ and $R[P, D] \subseteq S[L, C]$ require every P-value to lead to a tuple of Ref and a tuple of S that both carry the D-value. Since L is a key of Ref and Ref never changes, the C-value that a referenced entity of S must carry is fixed by its key. Hence every modification update is executable: write first the entities of S that gain referencing entities, then the entities of R, and last the entities of S that lose them. Writing first an entity of S that loses its referencing entities strips them of their witness, so such updates are not order-insensitive. The schema is order-sufficient but not order-insensitive, and $\mathcal{G} \not\models \Sigma$.

Two INDs into one relation can force a cycle of four writes although each is harmless alone. Let $T(P, V)$ be a frozen reference table with key P, let $R(K, A, D)$ and $Q(L, B, G)$ have frozen keys and the frozen attributes D and G, and let $S(C, E, F)$ have the frozen key C. The INDs are $R[AD] \subseteq T[PV]$, $Q[BG] \subseteq T[PV]$, $R[AD] \subseteq S[CE]$ and $Q[BG] \subseteq S[CF]$. Each IND into S alone behaves as in Ex. A.9. Together, let T hold $(1, 0)$ and $(2, 0)$, let r move from 1 to 2 and q from 2 to 1, and let s_1 and s_2 with keys 1 and 2 change (E, F) from $(0, 1)$ to $(1, 0)$ and from $(1, 0)$ to $(0, 1)$. Then r waits for s_2, s_2 for q, q for s_1, and s_1 for r, so no order exists. We conjecture that the condition $\mathcal{G} \models \Sigma$ remains exact for acyclic INDs if no pointer contains the primary key of its own relation, no IND derives a key, and no IND transfers an FD from frozen reference data. The evidence for this conjecture compares the condition with the semantics. The semantics enumerates small modification updates, and a SAT search over larger updates covers the cases in which the condition fails but enumeration finds no cycle. A random generator combines composite pointers, pointers that meet the primary key, attributes shared by two INDs and chains of copies. It confirms the decider of class $\mathcal{K}$ on all 33,445 pairs inside the class, and the condition and the semantics agree on all 5,400 pairs in the class of the conjecture. Outside this class, the condition fails while write order suffices on 1,039 of 46,556 pairs, and each of them has a pointer that contains the primary key of its relation or ranges over

frozen reference data. In all but 10 of these pairs some update is not order-insensitive. In the remaining ones a frozen copy fixes the attribute it copies in every referenced entity, so every update is order-insensitive although $\mathcal{G} \not\models \Sigma$.

A.4 Proofs for Sec. 5

The conflict graph rewrites condition (4) of Thm. 4.5 as a set of binary clauses.

Proof of Thm. 5.1. Freezing $Z \subseteq W$ leaves the attributes in $W \setminus Z$ mutable. For an IND $R[PD] \subseteq S[LC]$, condition (4) of Thm. 4.5 is equivalent to the conjunction of the clauses "p or c is frozen" for all $p \in P$ and $c \in C$, and "C_i or D_i is frozen" for all positions i. Indeed, if $C \subseteq Fz$, every clause holds. Otherwise, some $c \in C$ is mutable, the first clauses freeze P, and the second clauses are the rest of condition (4). A clause that mentions an attribute outside W holds, and the other clauses are the edges of H. Hence Thm. 4.5 makes Σ order-sufficient exactly when Z covers every edge of H.

For the hardness, let G be a graph whose edges are oriented along a numbering of its nodes, so that the orientation is acyclic. For every node u, the relation R_u has the primary key K_u, the attribute A_u, and an attribute D_{uv} for every edge from u to v. Every such edge contributes the IND $R_u[A_u D_{uv}] \subseteq R_v[K_v A_v]$. The schema belongs to $\mathcal{K}$: the pointer A_u is shared by the outgoing edges of u and is disjoint from K_u, every copy D_{uv} lies outside K_u and occurs in one IND only, and every source A_v lies outside K_v and is no copy. With $W = \{A_u\}$ and unit weights, H has the edge $\{A_u, A_v\}$ for every edge of G and no other edge, since the copies are frozen. MIN-FREEZE with bound k then asks whether G has a vertex cover of size at most k, which is NP-hard [38]. □

The tractable case of Sec. 5.1 follows from the same graph. Every edge of H joins attributes of two relations that an IND connects, so a 2-coloring of the undirected IND graph colors H. A minimum-weight vertex cover of a bipartite graph is a minimum cut of the standard flow network. The following example weighs the minimal freezes of the running example.

Example A.10 (Minimal freezes). In Ex. 4.6, H has an edge between *Project*.*gid* and *path* and one between *path* and *gpath*. Its minimal vertex covers are $\{path\}$ and $\{Project.gid, gpath\}$. Freezing *path* requires rewriting every rename, whereas freezing *gid* and *gpath* requires rewriting every move and every rename of a group with projects. Hence freezing *path* is cheaper unless renames of empty groups outnumber moves.

For shard keys, sufficiency reuses Lem. A.2, and necessity reuses the two-tuple constructions of Thm. 3.6.

Proof of Thm. 5.2. *Sufficiency.* Write the shards one at a time, each in one atomic step and in any order. Assume that two tuples t and u violate an elementary FD $X \to A$ of Σ^+ in some intermediate state. By Lem. A.2, the FD is unstable. The tuples of one shard are all old or all new, and I and J satisfy Σ, so t and u lie on different shards. They agree on $X \cap Fz$, whose values never change, so their tuples in I agree on $X \cap Fz$ and hence on $(X \cap Fz)^+ \supseteq SK$. Since $SK \subseteq Fz$, the tuples t and u agree on SK and lie on the same shard, a contradiction.

Necessity. We first reduce the condition to two weaker ones. For every unstable elementary FD $X \to A$ in Σ^+, let (i) require $X^+ \supseteq SK$ if A is mutable, and let (ii) require $(X \setminus \{x\})^+ \supseteq SK$ for every mutable $x \in X$ if A is frozen. Assume (i) and (ii). We show $(X \cap Fz)^+ \supseteq SK$ by induction on $|X \setminus Fz|$, where an FD with a frozen right side precedes one with a mutable right side of the same size. Let $Y = X$ if A is mutable. Otherwise, A is frozen, X contains a mutable x because the FD is unstable, and we let $Y = X \setminus \{x\}$. Then $Y^+ \supseteq SK$ by (i) or (ii). Let $s \in SK$. If $s \in Y$, then $s \in X \cap Fz$ because $SK \subseteq Fz$. Otherwise, some $Y' \subseteq Y$ gives an elementary FD $Y' \to s$. If $Y' \subseteq Fz$, then $s \in (X \cap Fz)^+$. Otherwise, $Y' \to s$ is unstable, since a frozen key inside Y' would make $X \to A$ stable. It has a frozen right side and $|Y' \setminus Fz| \leq |Y \setminus Fz|$, so it precedes $X \to A$. The induction hypothesis gives $(Y' \cap Fz)^+ \supseteq SK$, and $s \in (X \cap Fz)^+$ because $Y' \cap Fz \subseteq X \cap Fz$.

It remains to refute (i) and (ii) by updates. If (i) fails, take two tuples whose agree set is $C = X^+$ and give both the same new value on A, as in Case 2 of the proof of Thm. 3.6. If (ii) fails, take two tuples whose agree set is $C = (X \setminus \{x\})^+$ and swap their mutable parts, as in Case 3. In both cases $SK \not\subseteq C$, so the two tuples differ on SK, and some placement puts them on different shards. Every execution by single-shard steps then passes through a state in which exactly one of them is new, and this state violates $X \to A$. □

The condition of Thm. 5.2 uses closure and not containment, as the following example shows.

Example A.11 (Closure, not containment). Let a relation (*Id*, *Zip*, *City*, *Tax*) have the key *Id* and the FDs $Zip \to City$ and $City \to Tax$, and let only *Tax* be mutable. The shard key *City* meets the condition for the unstable FD $Zip \to Tax$, since tuples that agree on *Zip* lie in the same city.

On the running example, the condition selects the shard key as follows.

Example A.12 (Sharding projects by group). Consider again Ex. 3.7, where *Project* stores the path under $gid \to gpath$, and let the workload rename groups but never move projects. The FD $gid \to gpath$ is unstable, and the condition requires $\{gid\}^+ \supseteq SK(Project)$, so $SK(Project) = \{gid\}$, the only frozen attribute in $\{gid\}^+$. Sharding projects by their group turns every rename into one single-shard step, whereas sharding them by *pid* requires atomic visibility.

For fresh updates, the proof of Prop. 3.8 shows that the two tuples of a violation agree on X already in I, and hence on X^+. Sufficiency then needs $X^+ \supseteq SK$ only for the elementary FDs that Prop. 3.8 retains, and necessity uses the construction for (i) alone.

Class $\mathcal{K}$ yields binary updates, and the proof of the next theorem reads the edges of $\mathcal{P}$ off the INDs.

Proof of Thm. 5.3. Let (I, J) be a modification update, let $\rho = R[PD] \subseteq S[LC]$ be an IND of Σ, and let r be a changed entity of R. As in the proof of Prop. 4.7, let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. The old tuple of r has a witness under ρ exactly when s_I is old or $J(s_I)[C] = I(r)[D]$, and the new tuple has one exactly when s_J is new or $I(s_J)[C] = J(r)[D]$. An unchanged entity of R keeps its witness, since its referenced entity has the same key in I and J, and I and J are legal. The primary keys are frozen and are the only FDs, so the update is binary, and the

graph whose edges lead from some r to s_I with $J(s_I)[C] \neq I(r)[D]$ and from some s_J to r with $I(s_J)[C] \neq J(r)[D]$ is a precedence graph.

Sufficiency. Let ρ satisfy condition (4). If $C \subseteq Fz$, every entity of S keeps its sources, and the two inequalities fail because I and J are legal. Otherwise $P \subseteq Fz$, so $s_I = s_J$, and at every position i the source C_i or the copy D_i is frozen. In both cases $I(s_I)[C_i] = I(r)[D_i] = J(r)[D_i] = J(s_I)[C_i]$, and again neither inequality holds. Hence every edge of $\mathcal{P}$ stems from an IND that violates condition (4) and therefore carries the shard key. Its relations share a group, and since I and J satisfy Σ, $I(r)[SK(R)] = I(s_I)[SK(S)]$ and $J(r)[SK(R)] = J(s_J)[SK(S)]$. Shard keys are frozen, so r, s_I and s_J agree on their shard-key values and lie on one shard under every placement. Every edge of $\mathcal{P}$ thus stays within one shard, so does every strongly connected component, and Lem. 5.5 (1) executes the update.

Necessity. Let ρ violate condition (4) and not carry the shard key. The update (I, J) of the proof of Thm. 4.5 for ρ has a cycle of $\mathcal{P}$ through an entity r of R and its witness s under ρ, namely $r \to s \to r$ in the first case and $r_1 \to s_1 \to r_2 \to s_2 \to r_1$ in the second. If R and S lie in different groups, some placement puts r and s on different shards. Otherwise Σ does not imply the equality of the shard keys, so some finite instance M of Σ has a tuple r' of R whose witness s' under ρ has $s'[SK(S)] \neq r'[SK(R)]$. FDs and INDs are preserved under direct products [28], so $I \times M$ and $J \times M$, with pairs of values encoded as constants, satisfy Σ, and they form a modification update, since M stays fixed. In the product, (r_k, r') references (s_l, s') under ρ in a state exactly when r_k references s_l in the corresponding state of (I, J), and the second components agree because s' is the witness of r'. Hence every edge between r_k and s_l lifts to one between (r_k, r') and (s_l, s'), and the cycle lifts to $(r, r') \to (s, s') \to (r, r')$, or to $(r_1, r') \to (s_1, s') \to (r_2, r') \to (s_2, s') \to (r_1, r')$. The shard-key values of (r, r') and (s, s') differ in the second component, so some placement puts them on different shards. In both cases a cycle of $\mathcal{P}$ spans two shards under some placement, and Lem. 5.5 (1) rules out every execution by single-shard steps. □

For fresh updates, the sufficiency part applies verbatim. The first case of the necessity part is a fresh update, also in the product, whose new source values are pairs with a new first component, so every IND with a mutable copy of a mutable source must carry the shard key. Neither bound is exact. An IND whose pointer and source are mutable and whose copy is frozen yields only edges from R to S under fresh updates, by the argument of the proof of Prop. 4.7, which applies to each IND separately. If it is the only IND that violates condition (4), every fresh update is executable entity by entity, so it need not carry the shard key. However, INDs can combine, for example two INDs between the same relations. Let $R(N, A, B, D, E)$ and $S(L, C)$ have the primary keys N and L, let only A, C and E be mutable, and let the INDs be $R[AD] \subseteq S[LC]$ and $R[BE] \subseteq S[LC]$, with $SK(R) = B$ and $SK(S) = L$ in one group. The second IND carries the shard key, and the first has a frozen copy. Let r_1 and r_2 reference s_1 and s_2 under the second IND and s_2 and s_1 under the first, and let every value of C, D and E be 1. The fresh update that gives s_1 and s_2 new values of C, gives r_1 and r_2 the matching values of E, and moves both under the first IND to an unchanged entity whose C-value is 1 has the cycle $r_1 \to s_2 \to r_2 \to s_1 \to r_1$. Since r_i lies on the shard of s_i, the cycle spans two shards under every placement that separates the keys of s_1 and s_2, and the update is not executable under SK. Combinations through a third relation exist as well, so an exact condition has to follow paths of INDs. A bounded SAT search confirms Thm. 5.3 on 13,199 combinations of 280 random schemas of class $\mathcal{K}$, freezes, shard keys of one attribute and groups. It finds no counterexample with up to five entities and five values wherever the condition holds, and a counterexample with at most four changed entities wherever it fails.

Insertions and deletions need no precedence graph when the INDs are acyclic.

Proof of Prop. 5.4. Let the update only insert entities, so that $I \subseteq J$, and write the relations so that S precedes R whenever Σ has an IND from R to S. Every intermediate state L satisfies $I \subseteq L \subseteq J$, and a subset of J satisfies every FD that J satisfies. Let t be a tuple of L in a relation R, and let $R[X] \subseteq S[Y]$ be an IND. If t is old, its witness in I lies in L. If t is new, J holds a witness s of t, and s is old or belongs to S, whose new tuples are all written before those of R, since $S \neq R$ by acyclicity. For deletions, the order passes through the states of the insertion from J to I in the reverse order, which writes the relations in reverse topological order. □

The block lemma reads the precedence graph as the complete list of violations.

Proof of Lem. 5.5. Since the update is binary, an intermediate state $I \oplus T$ is legal if and only if no edge of $\mathcal{P}$ leads from an entity outside T to an entity in T. (1) Order the strongly connected components of $\mathcal{P}$ topologically and write each in one step, which touches one shard. After every step, the written entities are closed under predecessors, so the state after every step is legal. Conversely, consider the first step that writes an entity of a component that spans two shards. This step writes only part of the component, so some edge of the component leads from an unwritten entity to a written one, and the state after the step is illegal. (2) If every shard commits exactly once, an execution is an order of the shards. It is legal if and only if every edge between two shards leads from an earlier shard to a later one. Such an order exists if and only if the graph obtained by merging the entities of every shard is acyclic. □

Rounds refine model (1) by letting the transactions of one round commit in any order.

Proof of Prop. 5.7. A state is legal if and only if no edge of $\mathcal{P}$ leads from an unwritten entity to a written one, since the update is binary. Let an execution in d rounds be legal, and let $r(e)$ be the round of the transaction that writes the entity e. Consider an edge from v to u. If $r(v) > r(u)$, the state after round $r(u)$ writes u but not v. If $r(v) = r(u)$ and u and v lie in different transactions, the transaction of u may commit first. Both states are illegal, so $r(v) \leq r(u)$, with equality only if u and v share a transaction and hence a shard. Along a path that ends in e, the round therefore grows by at least one at every edge between two shards, and $d \geq r(e) \geq 1 + \lambda(e)$.

Conversely, every cycle of $\mathcal{P}$ lies within a strongly connected component and hence within one shard. Removing a cycle from a walk therefore keeps all its edges between two shards, so $\lambda(e)$ is also

the largest number of such edges on a walk that ends in e. For an edge from v to u, a walk that ends in v extends to u, so $\lambda(v) \leq \lambda(u)$, and $\lambda(v) < \lambda(u)$ if the edge joins two shards. Hence v lies in the transaction of u or in an earlier round. After the transactions of the earlier rounds and any of the current round have committed, the written entities are therefore closed under predecessors, and every state is legal. □

A relay that validates what the update read faces the same constraints as the original rounds, and its locks protect it only from updates that respect them. Here an entity *e shares a constraint* with a tuple t of another entity if one of the two tuples, t or the tuple of e, agrees with the other on the left side of an FD or is a witness of the other under an IND. An entity shares a constraint with an entity u of an update if it shares one with the current, the old or the new tuple of u, and a relay reads an entity that is absent as absent. We call an update *fenced* if it changes an entity only under an exclusive lock and commits a new tuple only while it holds a lock on every other entity that shares a constraint with the tuple, taken before it read the entity. Under primary keys and INDs into them, these entities are the parents and the children of the tuple, and a witness is the entity that carries the referenced key, so a fenced update locks the parent of every child that it inserts or moves.

Proposition A.13 (Recovery). *Let Σ consist of primary keys and INDs into them, let the other updates be fenced and lead from legal states to legal states, and let a crash stop leveled commits after some round in a legal state. Let a relay commit the remaining rounds as a fenced update that, before round k commits, locks the entities of round k, then reads every entity that shares a constraint with one of them, and aborts unless all are as round $k-1$ left them. Then every state after a commit is legal, and the relay completes the update unless another update has changed one of these entities.*

An aborted relay leaves the update stopped after some round in a legal state, and its outbox tells the application which writes remain, so that it can plan them again on the current state or report the update as failed.

Proof of Prop. A.13. The crash leaves a legal state after some round i, which Prop. 5.7 ensures when no other update intervenes, and other updates lead from legal states to legal states, so it suffices to show that every commit of the relay does. If the relay aborts, none of its writes becomes visible. Otherwise, consider the commit of a round $k > i$, and let u be an entity of round k. The relay locks u exclusively before it reads the entities that share a constraint with u, and, being fenced, it holds a lock on the parents of the new tuple of u until round k has committed. Hence no other update changes these parents between the read and the commit. No other update commits a tuple of a child of u in this time either, since being fenced it would need a lock on u. A child that another update has moved away from u before the read was read as such, and one that it moves away after the read no longer shares a constraint with u. Hence, at the commit, every entity that shares a constraint with an entity of round k is as the relay read it, which is as round $k-1$ left it. An IND into a primary key is violated by a tuple and the entity that carries the referenced key, so every violation after the commit either involves no entity of round k, and then it exists before the commit, or involves an entity of round k and one that is as round $k-1$ left it. In the second case the violation also exists in the state after round k of the original execution, which is legal by Prop. 5.7. Hence the state after the commit is legal. The relay overwrites only entities that it has locked and found as the crash left them, so it overwrites no change of another update. □

Deadlocks need waits for locks, which the protocol of Sec. 5.3 excludes.

Lemma A.14 (No hidden deadlocks). *If every lock request of leveled commits, for a write, a fence or a validation, fails instead of waiting, and an update whose request fails aborts its open transactions, then no update waits for a lock while it holds one, and no deadlock arises.*

Proof. A transaction of leveled commits acquires locks only while it executes its statements, never waits for a lock, and releases its locks when it commits or aborts. Afterwards, an update waits only for the commits of its own earlier rounds, and a commit acquires no lock. A deadlock is a cycle of updates in which each waits for a lock that the next one holds, so none arises. □

Retries with randomized backoff keep updates that fail on the same lock from failing again in step, but they guarantee no progress: under heavy contention most attempts fail (Sec. 6.3), and a relay whose locks fail stops the update as an abort does.

A pause between rounds yields such a state for every fast reader, since it orders the commits of different rounds in time.

Proof of Prop. 5.8. Times are real times, and the clocks of the writer and the reader measure durations alike. For a round i, let a_i be the time at which the last transaction of round i acknowledged its commit. Every transaction of round i commits before a_i, and every transaction of a later round k commits after $a_{k-1} + \delta \geq a_i + \delta$, since its commit is sent no earlier. The reader reads every shard s at one time t_s and sees on s exactly the transactions of s that committed before t_s (assumption 1), and $|t_s - t_{s'}| < \delta$ for all shards s and s'. If the reader sees no transaction of the update, it sees the initial instance, which is legal. Otherwise, let j be the latest round of which it sees a transaction, on a shard s'. If $j > 1$, then $t_{s'} > a_{j-1} + \delta$, and every transaction of a round $i < j$, on a shard s, committed before $a_i \leq a_{j-1} < t_{s'} - \delta < t_s$. Hence the reader sees every transaction of the rounds before j, some of round j and none of the later rounds. By the proof of Prop. 5.7, the entities that these transactions write are closed under predecessors, so the state that the reader sees is legal. □

Replicas shift the times at which a reader sees a shard by at most their lag.

Corollary A.15 (Replicas). *Let every shard be read from a replica that applies the commits of the shard in their order, at most Δ after they happen. Then Prop. 5.8 holds for every reader that reads each replica once, and all replicas within less than $\delta - \Delta$.*

Proof. A replica read at time t shows a prefix of the commits of its shard that contains every commit before $t - \Delta$ and none after t. Hence it shows exactly the transactions of its shard that committed before some time t' with $t - \Delta \leq t' \leq t$. Read times within less than $\delta - \Delta$ of each other thus yield such times t' within less than δ of each other, and the proof of Prop. 5.8 applies to them. □

The reading of the precedence graph in the proof of Prop. 5.7 also proves that concurrent updates need only merge into a legal instance.

Proposition A.16 (Concurrency). *Let updates that change pairwise disjoint sets of entities start from a common instance, and let their combination, whose changed entities are the union of theirs, be binary. If applying all of them yields a legal instance, then every interleaving of legal orders of the updates passes only through legal instances.*

Proof of Prop. A.16. Applying all updates yields a legal instance, so their combination is an update, and let $\mathcal{P}$ be its precedence graph. Since the combination is binary, an intermediate state of the combination is legal if and only if no edge of $\mathcal{P}$ leads from an unwritten entity to a written one. Applying a single update alone yields its final instance, which is legal and is the intermediate state of the combination that writes exactly the changed entities of this update. Hence no edge of $\mathcal{P}$ enters the changed entities of an update from those of another. A state of an interleaving writes a prefix of the order of every update. An edge into a written entity u therefore starts at an entity of the same update, and since the prefix states of that update alone are legal, it starts within the written prefix. □

In the example of Sec. 5.3, leveled commits hold the lock on g_2 in their open transactions: the deleting update locks g_2 in the transaction of its last round before it reads the projects of g_2, and the inserting update locks g_2 for sharing in a transaction that commits after the project.

B FURTHER EXPERIMENTAL RESULTS

This appendix reports the results of Sec. 6 that the argument of the body does not need.

An eager first round. Aborting a failed update before any write is visible costs one round trip. On two or three rows, ordered commits of one row each commit every row in the message that executes it and take up to a quarter less time with one client, but a row that fails leaves the earlier rows visible. Where no write can fail logically, an eager first round saves this round trip and makes leveled commits the fastest way of writing with one client at every size (Fig. 4).

Fences and deadlocks. Concurrent updates that do not merge into a legal instance need the locks of Sec. 5.3. When clients added projects to groups that other clients merged, 24% of the merges left an orphan under leveled commits and 68% under one 2PC. With the locks, no merge did, merges took as long as before, and additions took 0.85 instead of 0.71 ms. Lock requests that fail instead of waiting keep these locks free of hidden deadlocks, at the price of retries. When 64 clients moved projects between hot groups, writes that waited closed cycles of waits across workers, which only a lock timeout broke, whereas writes that failed closed none and sustained 6,440 updates per second, against at most 35 for 2PC, whose waits the deadlock detection of Citus had to break. Yet 63.5% of their attempts failed, and their 99th percentile latency reached 184 ms. Locking the rows in one global order avoids the cycles as well and sustained 3,871 updates per second, and with 8 or 16 clients under milder skew, waits bounded by a timeout of 100 ms were faster than failing writes, so failing writes pay off only under heavy contention.

Unique keys under sharding. A shard key that makes updates executable may still split a unique key, which a shard then cannot check. In GitLab, 1,040 of the 1,127 tables have a single-column shard key that makes every fresh update executable and that every unique key of the table determines through copies, so that a shard checks the key from local data, as Gupta and Widom [36] verify global constraints. The rule of Citus [20], which requires the shard key inside the primary key and every unique key, admits a single-column shard key for only 527 tables. The other 87 tables hold globally unique names, paths and tokens, which need a check across shards under any placement.